



AUDIT REPORT

August 2026

For



Table of Content

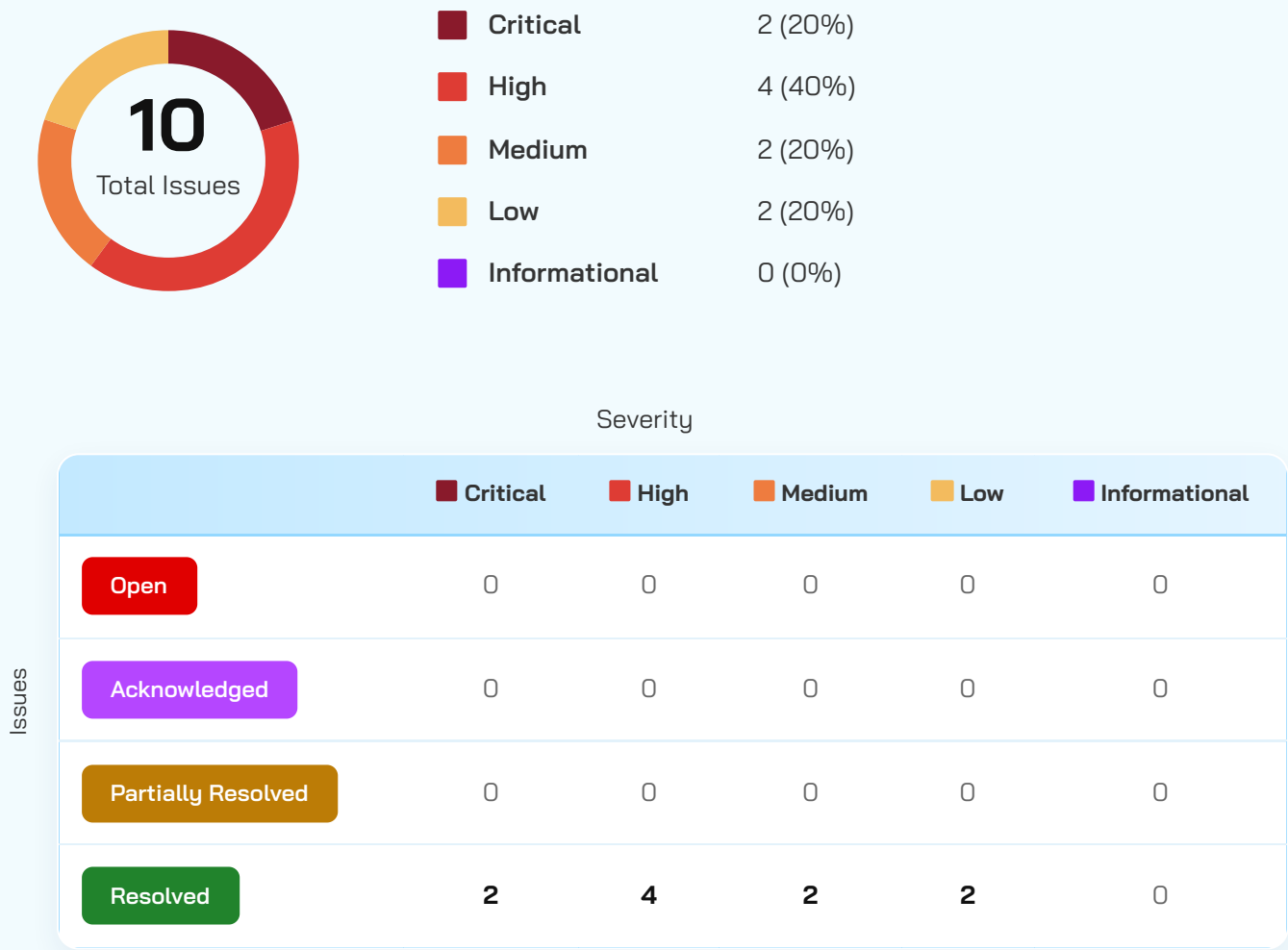
Executive Summary	04
Number of Security Issues per Severity	05
Summary of Issues	06
Security Areas Assessed	07
Techniques and Methods	08
Types of Severity	10
Types of Issues	11
Severity Matrix	12
■ Critical Severity Issues	13
1. Unauthenticated gRPC signer; caller-spoofable metadata selects the production MANAGER key	13
2. executeTx is an unauthenticated arbitrary-instruction / arbitrary-signer execution primitive	15
■ High Severity Issues	16
1. buildEd25519Ix is an unauthenticated arbitrary-message signing oracle over any KMS key	16
2. A single hot manager key is the sole on-chain authority to move the custody pool, with no amount or velocity cap at any layer	17
3. Mainnet V4 admin/manager/config authority is bound to a single hot KMS key co-located in the one signer keyring	18

4. Settlement idempotency and double-spend protection are unreliable (volatile cache, stale confirmation status, loose retry)	19
■ Medium Severity Issues	20
1. Deploy pipelines force-merge PRs with gh pr merge --admin and the actor restriction is commented out	20
2. No .dockerignore plus COPY . . bakes CI-generated .env and committed SSH host private keys into the published image	21
■ Low Severity Issues	22
1. Cloudflare WAF and edge controls are bypassable via directly reachable GKE gateway origins	22
2. Unauthenticated callers can inject Squads proposals pre-approved with the hot MANAGER vote (1-of-N)	23
Minor Hardening	24
Accepted Business / Design Decisions	27
Documentation versus Reality	28
Positive Controls Confirmed	29
Conclusion & Disclaimer	30

Executive Summary

Project Name	Zynk Protocol
Assessment Type	Operational Security (OPSEC) Assessment
Overview	The audit confirms two funding-gated Critical design risks in the fastlane signer: an unauthenticated insecure-bound gRPC service whose caller-spoofable metadata selects the production MANAGER key, and an executeTx/buildEd25519Ix pair that form unauthenticated arbitrary-instruction and arbitrary-message signing oracles over any KMS key. Blast radius is bounded today because the funded ~\$2.28M core (ZYNKctWoa) sits behind disjoint 3-of-4 Squads multisigs and an on-chain whitelist while the single-hot-key V4 core is ~unfunded, but it becomes unbounded once custody migrates to V4. Beyond the signer, real issues remain around single-hot-key custody authority with no on-chain cap/velocity, unreliable settlement idempotency, and CI/CD force-merge and baked-secret hygiene (both Medium). The remaining two dozen items are largely defense-in-depth, unreachable, or fail-closed hygiene that collapse into hardening notes, plus several accepted-operator tradeoffs reclassified as business decisions.
Assessment Scope	Fastlane signing service and SDK, signer / KMS key-management model, on-chain zynk-core (funded core ZYNKctWoa, V4 core ZYNKuSAu) and zynk-orbit configuration, GitHub Actions CI/CD pipelines, container build, and Cloudflare / GKE edge infrastructure.
Repositories	github.com/Zynklabs/fastlane github.com/Zynklabs/fastlane-sdk
Blockchain	Solana (with EVM token support)
Method	Manual Code Review, On-chain Configuration Verification, CI/CD & Infrastructure Review, External Attack Surface Testing
Findings	2 Critical · 4 High · 2 Medium · 2 Low (consolidated from 47; 26 minor items folded into a hardening list, 6 accepted as business decisions)
Report Date	12 August 2026
Fixed In	fastlane/tree/cleanup fastlane-sdk/tree/zynk-core-v4.2+orbit-v2 fastlane/commit/2338726e fastlane/commit/c92748f9

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Unauthenticated gRPC signer; caller-spoofable metadata selects the production MANAGER key	Critical	Resolved
2	executeTx is an unauthenticated arbitrary-instruction / arbitrary-signer execution primitive	Critical	Resolved
3	buildEd25519Ix is an unauthenticated arbitrary-message signing oracle over any KMS key	High	Resolved
4	A single hot manager key is the sole on-chain authority to move the custody pool, with no amount or velocity cap at any layer	High	Resolved
5	Mainnet V4 admin/manager/config authority is bound to a single hot KMS key co-located in the one signer keyring	High	Resolved
6	Settlement idempotency and double-spend protection are unreliable (volatile cache, stale confirmation status, loose retry)	High	Resolved
7	Deploy pipelines force-merge PRs with gh pr merge --admin and the actor restriction is commented out	Medium	Resolved
8	No .dockerignore plus COPY . . bakes CI-generated .env and committed SSH host private keys into the published image	Medium	Resolved
9	Cloudflare WAF and edge controls are bypassable via directly reachable GKE gateway origins	Low	Resolved
10	Unauthenticated callers can inject Squads proposals pre-approved with the hot MANAGER vote (1-of-N)	Low	Resolved

Security Areas Assessed

The following operational security areas were reviewed across the application, on-chain, and infrastructure layers:

✓ Service Authentication & Authorization

✓ Caller Identity & Metadata Trust

✓ Transport Security (TLS / mTLS)

✓ Network Exposure & Segmentation

✓ Signing Oracle Exposure

✓ KMS Key Management & Keyring Segregation

✓ Hot-Wallet Custody Controls

✓ Amount & Velocity Limits

✓ Multisig Governance & Separation of Duties

✓ On-chain Config Authority

✓ Settlement Idempotency & Retry Safety

✓ Pause & Circuit Breakers

✓ CI/CD Merge & Release Controls

✓ Branch Protection & Environment Gates

✓ Supply-Chain & Action Pinning

✓ Container Build Hygiene

✓ Secret Management

✓ Logging & Credential Leakage

✓ Cloud IAM & Workload Identity

✓ Edge / WAF / Origin Protection

✓ Dependency Scanning

✓ Social Engineering & Signer OPSEC

Techniques and Methods

Throughout the operational security assessment, care was taken to ensure:

- Signing and money-movement surfaces are authenticated and authorised per caller.
- Private key material and signing authority follow least privilege and separation of duties.
- On-chain custody configuration matches the documented trust model.
- Settlement flows remain idempotent and safe under retries, congestion and infrastructure faults.
- Build and deployment pipelines cannot ship unreviewed or tampered code.
- Secrets are not baked into images, logs or source control.
- Externally reachable infrastructure cannot bypass edge protections.

The following techniques, methods, and tools were used to review the in-scope systems:

Architecture & Trust-Boundary Review

Mapped the trust boundaries between the fastlane signing service, the KMS keyring, the Squads multisigs and the on-chain zynk-core / zynk-orbit programs to identify where signing and custody authority is concentrated and which controls actually bound it.

Source Code Review

Manual review of the fastlane service and SDK covering gRPC transport and binding, caller identity derivation, signer resolution, transaction relaying, caching, confirmation and retry logic, and configuration and secret loading.

On-chain Configuration Verification

Decoded live program configuration and role addresses for the funded core and the V4 core to confirm admin, guardian, manager and attester authority, multisig thresholds, whitelist and pause controls.

CI/CD, Build & Infrastructure Review

Reviewed GitHub Actions workflows, merge and environment gates, Dockerfile and build context, secret handling, action pinning, workload identity usage and logging paths.

External Attack Surface Testing

Probed public endpoints and origin reachability, including Cloudflare edge bypass via direct origin resolution and origin discovery through Certificate Transparency logs.

Tools and Platforms Used for Assessment

Manual code review, curl, Certificate Transparency logs, Solana RPC and Anchor IDL decoding, custom recon scripts, GitHub Actions workflow analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are weaknesses that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of funds, compromise of signing keys or custody authority, a takeover of critical infrastructure, or other catastrophic failures. Critical issues are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses in applications, infrastructure, key management or operational processes that could result in significant financial loss, major service disruption, or substantial compromise of the protocol's intended operations. Exploitation may require specific conditions or a moderate level of skill, but the potential damage is considerable. These findings should be resolved before the affected systems are relied upon in production.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity issues are gaps that could lead to moderate financial loss, exposure of sensitive material, or partial disruption of operations. Exploiting them typically requires more specific or less common conditions, such as a pre-existing privileged position, and the overall impact is lower than High or Critical issues. They should still be addressed to strengthen the overall security posture.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are minor weaknesses or defense-in-depth gaps with limited impact on funds or core operations. Exploitation would require unlikely scenarios or yield minimal gain for an attacker. While they pose no immediate threat, addressing them when feasible reduces the attack surface and the chance that they combine with other issues.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings are not security vulnerabilities in the traditional sense. They highlight opportunities to improve configuration hygiene, documentation, monitoring, logging, or adherence to operational best practices. They pose no immediate risk but offer valuable insights for long-term resilience and maintainability.

Types of Issues

Open Security vulnerabilities identified that must be resolved and are currently unresolved.	Resolved These are the issues identified in the initial audit and have been successfully fixed.
Acknowledged Vulnerabilities which have been acknowledged but are yet to be resolved.	Partially Resolved Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		■ High	■ Medium	■ Low
Likelihood	■ High	Critical	High	Medium
	■ Medium	High	Medium	Low
	■ Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

Unauthenticated gRPC signer; caller-spoofable metadata selects the production MANAGER key

Resolved

Category

APPLICATION

Location

```
src/index.ts:68,77-78; src/utils/index.ts (getInitiator); src/interceptor.ts (dead);  
src/impls/base.ts, core.ts, orbit.ts
```

Description

The fastlane signing service exposes an insecure, unauthenticated gRPC endpoint whose only identity signal is client-supplied metadata, so any network-reachable peer can pick both its identity and the KMS signing key. This is the root access-control failure that makes every other fastlane money-movement primitive reachable.

Issue

src/index.ts:68/77-78 binds new `grpc.Server()` on 0.0.0.0:50051 with `ServerCredentials.createInsecure()` and registers no interceptor; the `interceptor/ipGuard/withIpGuard` are dead code (commented out, never imported). `getInitiator` (src/utils/index.ts) trusts spoofable `z-caller/z-env` metadata (allowlist commented) and defaults non-preprod callers to `manager=MANAGER`, the prod hot signer. All money-moving RPCs (transfer, collect, disburse, createOrder, replenish) inherit this: they are wrapped only in `withErrorHandler` and perform no caller-to-partner authorization.

Steps to Reproduce

From a peer with network reach to the pod, send any gRPC request to 0.0.0.0:50051 with arbitrary `z-caller/z-env` metadata; the service accepts it and resolves the prod MANAGER KMS signer.

Impact

Any reachable caller invokes signing/value-movement RPCs as the production manager. Blast radius is funding-gated today (funded core ZYNKctWoa is bounded by disjoint 3-of-4 Squads multisigs + on-chain whitelist; single-hot-key V4 core ~unfunded), but unbounded once custody moves to V4. Only remaining controls are NetworkPolicy and the on-chain whitelist.

Remediation

Terminate the insecure bind; require mTLS/authenticated transport and a real per-caller authorization interceptor (re-enable and enforce the allowlist). Never derive signer identity from client metadata. Enforce NetworkPolicy so only trusted callers reach the port. Add per-request amount ceilings and per-window velocity limits on transfer/collect/disburse.

Unauthenticated gRPC signer; caller-spoofable metadata selects the production MANAGER key *(continued)*

Resolution

Status: Resolved

With the arbitrary-input functions from #2 and #3 removed and unnecessary roles stripped out, allowing mutable z-caller and z-env metadata is an intentional architectural decision. This metadata is utilized solely for environmental routing rather than strict validation checks. Security is uncompromised, as signer authority is strictly enforced by the on-chain program config as the single source of truth.

Fixed In

<https://github.com/Zynklabs/fastlane/tree/cleanup>

<https://github.com/Zynklabs/fastlane-sdk/tree/zynk-core-v4.2%2Bborbit-v2>

Critical Severity Issues

executeTx is an unauthenticated arbitrary-instruction / arbitrary-signer execution primitive

Resolved

Category

APPLICATION

Location

```
src/impls/base.ts:230; src/modules/Base.ts:372-381; src/services/TxRelayer.ts:230-241;  
src/config/signers.ts
```

Description

The executeTx RPC forwards caller-supplied instructions and caller-named signer keys verbatim to the relayer, which resolves each name to a KMS key and signs. It is the single most dangerous RPC: an unauthenticated arbitrary-signing oracle over any key including the V4 admin authority.

Issue

impls/base.ts:230 executeTx (withErrorHandler only; never reads call metadata) -> modules/Base.ts:372 passthrough -> TxRelayer.ts:230-241 resolves each caller-named string via getKmsSigner and signs the caller's arbitrary instructions. config/signers.ts loads all 10 KMS keys with no allowlist; getOrCreateAta (base.ts:215) is a related unauth primitive that makes MANAGER pay unrecoverable rent to attacker-owned ATAs.

Steps to Reproduce

Call executeTx over the unauthenticated endpoint with attacker-chosen instructions and signerKeys= ("MANAGER") (or "MANAGER_ORBIT"); the relayer signs and submits.

Impact

Arbitrary transactions signed as any named key. Direct raw-transfer blast radius is small today (hot wallets ~0; funded pool is manager-gated + 3-of-4-whitelist-bounded), but the same oracle can sign as the unbounded single-key V4 admin (CRYpBZS8), cast multisig approvals, and forge attestations. Critical as a primitive; unbounded once custody moves to V4.

Remediation

Remove executeTx or restrict it to a fixed allowlist of vetted instructions; never accept caller-named signer keys. Authenticate the surface (see anchor finding). Use a dedicated rent-payer distinct from the settlement signer for ATA creation.

Resolution

Status: Resolved

executeTx method removed.

Fixed In

<https://github.com/Zynklabs/fastlane/commit/2338726e7894dd12679a0dbbc62fe05d62a98898>

High Severity Issues

buildEd25519Ix is an unauthenticated arbitrary-message signing oracle over any KMS key

Resolved

Category

APPLICATION

Location

```
src/impls/base.ts:92-105; src/config/signers.ts:49-62; src/utils/web3/solana.ts;  
src/modules/Core/index.ts:465,479
```

Description

buildEd25519Ix lets an unauthenticated caller choose both the KMS key and the raw message and returns a detached ed25519 signature with no domain separation, defeating the attester as an independent factor.

Issue

impls/base.ts:92-105 returns a raw ed25519Sig over caller-chosen bytes; config/signers.ts:49-62 .toUpperCase()s the prefix with no allowlist (ATTESTER/MANAGER/ORBIT selectable); utils/web3/solana.ts signs the raw TextEncoder bytes with no typed-message/domain separation. Core/index.ts threads ed25519Sig into pullAndCreateOrder with SYSVAR_INSTRUCTIONS, so the signature is verified on-chain as a pull authorization.

Steps to Reproduce

Call buildEd25519Ix with signer="ATTESTER" and an attacker-chosen message; receive a valid detached signature usable off the transaction path.

Impact

Replayable forged signatures over arbitrary bytes (attestations, cross-chain messages, order envelopes) usable entirely off the transaction path. Direct fund-movement rides the same funding/multisig-gated path as executeTx, so standalone value is the integrity/forgery break.

Remediation

Add a key allowlist, require typed messages with domain separation, bind order_id/amount/nonce/expiry into the signed payload, and never return raw detached signatures.

Resolution

Status: Resolved

buildEd25519Ix method removed.

Fixed In

<https://github.com/Zynklabs/fastlane/commit/2338726e7894dd12679a0dbbc62fe05d62a98898>

High Severity Issues

A single hot manager key is the sole on-chain authority to move the custody pool, with no amount or velocity cap at any layer

Resolved

Category

ON-CHAIN

Location

src/impls/core.ts; src/modules/Core/index.ts; on-chain core ZYNKctWoa Config

Description

createOrder/replenish/transfer/attestOrder each build fund-moving instructions signed by the single hot [manager] KMS key with no fastlane-side or in-program amount, authorization, or velocity check. This is active today on the funded ~\$2.28M core, guarded only by has_one=manager and a not-paused check.

Issue

src/impls/core.ts signs value-moving ixes with [manager] via TxRelayer with zero authz/ceiling/velocity, while whitelist/toggle/revoke correctly route through Squads multisig. manager = MANAGER/MANAGER_ORBIT hot key resolved via spoofable getInitiator. No on-chain per-tx cap, rolling velocity, or second-signer on egress. (Also: a single-key pause enables a recoverable settlement-outage DoS.)

Impact

Full-pool egress to whitelisted destinations bounded only by the Squads-gated whitelist; theft to a fresh attacker address additionally requires compromising the disjoint 3-of-4 whitelist authority. Automated ~1-tx/5-min settlement structurally requires the hot key, so egress outruns human response.

Remediation

Add an in-program per-tx cap + rolling velocity limit + short timelock/second-signer on egress above a threshold; sweep the majority to a cold/multisig reserve; add app-layer per-request ceilings and rate limits.

Resolution

Status: Resolved

Zynk Labs Team Comment

A single hot manager key (KMS generated) is an essential core architectural constraint and cannot be substituted with any other mechanisms. However, to enhance operational guardrails, we intend to introduce tx caps and rate limits in our next releases.

High Severity Issues

Mainnet V4 admin/manager/config authority is bound to a single hot KMS key co-located in the one signer keyring

Resolved

Category

ON-CHAIN

Location

```
src/config/signers.ts:15-17; src/config/kms/KmsSigner.ts:76; src/modules/Core/index.ts  
(beforeInit); on-chain V4 core ZYNKuSAu
```

Description

On mainnet the V4 admin/manager/guardian/attester all collapse to one hot KMS key (CRYpBZS8 = MANAGER_ORBIT), which lives in the single production keyring alongside every other signer, contradicting the org's own proven 3-of-4 posture on the funded program.

Issue

signers.ts:15-17 forces MANAGER_ORBIT -> SOLANA_MAINNET_ADMIN_WALLET_PK on mainnet; KmsSigner.ts:76 places every signer under the one solana-byok-ring; Core/index.ts beforeInit bootstraps V4 via initialize()/whitelistBeneficiary() authorized by MANAGER_ORBIT and actively pulls partner balances toward V4. The V4 config key is reachable via the unauthenticated executeTx/buildEd25519 primitives.

Impact

Single point of failure for all V4 custody roles with no keyring segregation. Funding-gated today (V4 ~\$0; \$2.28M on the multisig program), but the active V4 migration wiring makes the drain near-term rather than hypothetical.

Remediation

Introduce on-chain separation-of-duties (distinct multisig admin/guardian/attester as on the funded core) before funding V4; segregate keyrings per role/environment; keep the automation manager key least-privileged.

Resolution

Status: Resolved

Redundant roles have been removed from Zynk systems (both on-chain and off-chain), Plus, the Orbit components now depend on the Core config.

Fixed In

<https://github.com/Zynklabs/fastlane/commit/c92748f93dcfb25c1207075aff0200c05ea15c>

High Severity Issues

Settlement idempotency and double-spend protection are unreliable (volatile cache, stale confirmation status, loose retry)

Resolved

Category

APPLICATION

Location

```
src/utils/Cache.ts:24-27,41-44,58-67; src/utils/TxDecoder.ts:72-77;  
src/services/TxRelayer.ts:294-302,356-387
```

Description

The confirm-and-retry path resends a fresh, independently-valid signed transaction guarded only by a status check that reads a volatile, clearable, 100-minute-stale LRU cache, and process-local idempotency keyed on a caller-supplied (defaultable-empty) requestId. Two independent double-spend vectors exist.

Issue

Cache.ts:24-27 ttl 100min + allowStale + updateAgeOnGet (a frozen negative can persist indefinitely);
Cache.ts:41-44 clears the whole LRU at heap>0.8. getTxStatus polls only ~3s and its null result is cached;
TxRelayer #checkStatus treats null as 'did not land' and the block-height-exceeded retry (incl. JSON.stringify(error).includes) resends. requestId idempotency is write-after-confirm, process-local, in the clearable LRU, empty default = no idempotency.

Impact

Duplicate payment of customer funds under normal congestion/RPC lag/heap pressure/caller collision. Realized blast radius scales with settlement volume through the relayer.

Remediation

Persist idempotency in a durable store with a pre-send reservation (reserve-before-sign), require a non-empty requestId, treat null/unknown status as non-terminal (never as 'not landed'), and tighten the block-height-exceeded match.

Resolution

Status: Resolved

Zynk Labs Team Comment

Caching was introduced just for the sake of quick access of states and already executed transactions. It was never meant for idempotency. After removal of the arbitrary-input functions, all major functions depend on the on-chain state (mainly PDAs) as the ultimate source of truth.

Medium Severity Issues

Deploy pipelines force-merge PRs with gh pr merge --admin and the actor restriction is commented out

Resolved

Category

INFRASTRUCTURE

Location

```
.github/workflows/deploy_prod.yml:13,103;  
.github/workflows/staging_deploy_pipeline.yml:96,101
```

Description

Both prod and staging deploy workflows force-merge with --admin, the prod actor guard is commented out, and staging has no environment/actor gate and a commented-out Jira gate, so unreviewed or scan-optional code can reach the fastlane image that holds hot KMS keys.

Issue

deploy_prod.yml:103 and staging_deploy_pipeline.yml:101 both gh pr merge --merge --admin; deploy_prod.yml:13 actor guard commented; staging has no environment gate (prod has environment:production at :16) and its Jira exit 1 is commented (:96). Snyk and Claude review are advisory PR checks; GH_TOKEN is default GITHUB_TOKEN.

Impact

Supply-chain path into the fastlane hot-signer image, contingent on an unverified org-side bypass-actor precondition and funding-gated crown-jewel impact. No designated release engineer.

Remediation

Remove --admin force-merge; require reviewed merges via the production environment's required reviewers; add an enforced actor/branch gate on staging; make the Jira and Snyk gates blocking for deploy.

Resolution

Status: Resolved

Removed --admin force-merge. Added reviewers as required along with dependency and code scan checks with enforced actor.

Medium Severity Issues

No .dockerignore plus COPY . . bakes CI-generated .env and committed SSH host private keys into the published image

Resolved

Category

INFRASTRUCTURE

Location

Dockerfile:13; .github/workflows/deploy_prod.yml:129,179; repo-root host.key, host_ed25519.key

Description

The single-stage build copies the whole workspace, baking a CI-written .env and unencrypted committed SSH host private keys into the image layers in the private registry.

Issue

No .dockerignore; Dockerfile:13 COPY . .; deploy_prod.yml:129 writes Firestore config to .env then :179 docker build on the same workspace. host.key/host_ed25519.key are real unencrypted ed25519 keys at repo root, tracked (.gitignore lacks *.key). Baked .env is mostly recon/public (KMS resource names, public program IDs); the sensitive item is a possible provider key in SOLANA_RPC_URL. The SSH keys serve sshx.ts, which is not deployed (spawnSSH() commented + non-prod gated).

Impact

Secret-hygiene defect: image contains baked config and committed private keys. Exploitation needs an already-privileged registry position; the material risk is a possible RPC provider key and long-lived committed host keys.

Remediation

Add a .dockerignore, purge and rotate the SSH keys from git history, mount secrets at runtime (not baked), and use a multi-stage build copying only dist.

Resolution

Status: Resolved

Added .dockerignore for the below along with multistage docker build:

```
.git
*.key
host.key
host_ed25519.key
node_modules
README.md
```

Low Severity Issues

Cloudflare WAF and edge controls are bypassable via directly reachable GKE gateway origins

Resolved

Category

INFRASTRUCTURE

Location

GKE/GCLB origin 34.160.13.118 and the four public backends; Cloudflare edge for app.zynklabs.xyz

Description

The GKE/GCLB origins answer directly on public IPs, bypassing the Cloudflare edge, and the CT-logged origin cert makes discovery trivial. No auth bypass is demonstrated; only the compensating edge layer is lost.

Issue

POC: curl --resolve to the origin IP returns HTTP/2 200 with 'via: 1.1 google' and no cf-ray, vs control app.zynklabs.xyz 403 server:cloudflare + cf-ray. Backends lack a Cloud Armor CF-egress allowlist or Authenticated Origin Pulls; CT-logged LE cert (CN app.zynklabs.xyz) reveals origins.

Steps to Reproduce

curl -k --resolve app.zynklabs.xyz:443:34.160.13.118 https://app.zynklabs.xyz/ ; observe 200 with via:1.1 google and no cf-ray.

Impact

Loss of the edge defense-in-depth layer (WAF managed rules, bot challenge, L7 rate-limiting, IP/geo reputation). Reachability only; no demonstrated C/I/A impact. Real controls sit at the identity/app and on-chain layers.

Remediation

Enforce Authenticated Origin Pulls or a Cloud Armor Cloudflare-egress allowlist on all four backends and replace the CT-logged origin cert with a Cloudflare Origin CA cert.

Resolution

Status: Resolved

1. Network-layer allowlisting (Cloud Armor): All four backends now enforce a Cloud Armor security policy that only permits traffic sourced from Cloudflare's published IP ranges (IPv4 + IPv6); all other sources receive a default 403 Deny.
2. Mutual TLS / Authenticated Origin Pulls: Origins now cryptographically verify that the connecting client is Cloudflare, via a zone-scoped client certificate presented during the TLS handshake and validated at the GKE Gateway. Direct-IP requests without this certificate are rejected at the connection level, independent of the IP check above.

Low Severity Issues

Unauthenticated callers can inject Squads proposals pre-approved with the hot MANAGER vote (1-of-N)

Resolved

Category

ON-CHAIN

Location

```
src/impls/core.ts:526,546,566; src/impls/orbit.ts:182,202;  
src/modules/Squads/SquadsV4.ts:52-108
```

Description

whitelist/toggle/revoke beneficiary and LP operations pass caller-controlled args into Squads.createTx, which atomically bundles create+approve signed by MANAGER, so an unauthenticated caller injects a proposal already carrying one live MANAGER vote.

Issue

core.ts:526,546,566 and orbit.ts:182,202 pass caller args into Squads.createTx; SquadsV4.createTx:52-108 bundles vaultTransactionCreate + proposalCreate + proposalApprove(member=MANAGER), all MANAGER-signed. Reachable via the unauthenticated endpoint.

Steps to Reproduce

Call whitelistBeneficiary over the unauthenticated endpoint with attacker-chosen args; a Squads proposal is created carrying one MANAGER approval.

Impact

On the funded core, 2 further independent 3-of-4 sign-offs are still required, so marginal harm is proposal/rent spam (MANAGER is rentPayer) plus a social-engineering assist against signers who trust fastlane only enqueues legit proposals. Moot on the single-key V4 core where the direct primitives dominate.

Remediation

Authenticate proposal creation, use create-for-independent-review (no pre-seeded self-approval) for sensitive admin ops, and rate-limit.

Resolution

Status: Resolved

Zynk Labs Team Comment

Intended design decision. We accept the operational tradeoffs of this model, as risk is explicitly mitigated by requiring dual-human authorization within a 3-of-4 threshold multisig prior to transaction execution. Each multisig transaction request raised is carefully scrutinized before signing and execution.

Minor Hardening

Merged and low-signal items. These do not warrant standalone findings but are recommended as defense-in-depth improvements.

No.	Observation	Recommendation
1	getOrCreateAta (base.ts:215) lets an unauth caller make MANAGER pay ~0.002 SOL unrecoverable rent to attacker ATAs.	Use a dedicated rent-payer; strictly dominated by executeTx.
2	Manager operational key appears reused as an Operations multisig signer, shaving one signature off the 3-of-4 (composition claim unverified).	Keep automation keys out of the governance quorum.
3	Blind-signing / signing-interface integrity is the single barrier on a healthy 3-of-4.	Enforce clear-signing hardware, mandatory proposal simulation/decode, and out-of-band confirmation; consider separating guardian from program upgrade authority.
4	Unauthenticated sshx remote-shell (sshx.ts, port 2222) ships in the signer build though spawnSSH() is commented + non-prod gated.	Remove it and the committed host keys from the production image.
5	Concurrent createTx calls race on transactionIndex (SquadsV4.ts:42-43, SquadsV3.ts:38-45) and the per-signer TxChannel queue is dead code.	Serialize createTx per-multisig (Squads PDA uniqueness makes the loser fail atomically).
6	Caller-controlled values are concatenated unescaped into Squads proposal memos (core.ts:528/548/568, orbit.ts:184/204).	Escape/normalize and instruct signers to verify decoded instruction data, not the memo label.
7	rentPayer=MANAGER (SquadsV4.ts:56) draws recoverable rent from the sole hot signer.	Use a dedicated rent-payer plus rate limiting.
8	resolveKeyVersionName (KmsSigner.ts:22-28) has no project/ring allowlist though not caller-reachable today.	Pin ring/project/location to a constant allowlist.
9	getVaults env-separation filter (base.ts:538-547) is a no-op (missing return, fails closed) and keys on spoofable initiator.	Add the return or remove the dead control.
10	EVM getTokenDetails hardcodes decimals:6 (evm.ts:39, Evm/index.ts:36).	Source decimals per token/chain from the registry (only USD1/RLUSD on Ethereum are 18-decimal).

No.	Observation	Recommendation
11	generateHashedArray (crypto.ts:11-15) decodes a hex digest as UTF-8, halving PDA-seed entropy.	Use Buffer.from(hash,'hex') with a coordinated on-chain change (public seeds, no security impact today).
12	Top-level await fetch to CoinGecko at import (external.ts:17-23) makes a third-party a hard startup dependency for the signer.	Fetch lazily with a timeout and a hardcoded fallback currency allowlist.
13	Dockerfile uses npm install (not npm ci) and ships devDependencies + protoc toolchain on a mutable node:20-alpine tag.	npm ci, --omit=dev, multi-stage, digest-pin base.
14	Fatal config/secret-fetch failures call process.exit() with no code (envt.ts:17,56,62 -> exit 0), showing green to exit-code orchestrators.	Use process.exit(1).
15	GCP Secret Manager path is inert (envt.ts .env.sample header never matches; getCustomSecret unused) so all config loads from plaintext env.	Fix the parser and move SOLANA_RPC_URL into Secret Manager if it carries an API key.
16	Production-logging hygiene: TxDecoder.decodeEvent (TxDecoder.ts:142-144) logs full tx + credentialed RPC URL, staging logs the full Jira API response (staging:90-91), and CrashLoopBackOff kubectl logs are POSTed unredacted to Slack (deploy_prod.yml:221-234).	Gate/redact debug logs and credential-bearing URLs.
17	CI/CD supply-chain pinning: Snyk actions on @master (snyk_*_scan.yml:20) and mutable major tags on the id-token:write deploy job (checkout@v4, auth@v2).	SHA-pin the WIF-privileged deploy-job actions and verify the WIF attribute_condition pins exact repo + protected ref + environment.
18	Claude code-review workflow is granted unused id-token:write (claude-code-review.yml:15) and is prompt-injectable but tightly scoped (pull_request, narrow tool allowlist, no merge/approve).	Remove the unused permission.
19	Production secrets are fetched into a plaintext .env on an ephemeral runner with best-effort ::add-mask:: and the env-reader SA is likely database-wide.	Segregate env secrets per-environment and use native secret mounts with short-lived scoped creds.

No.	Observation	Recommendation
20	On-chain governance/SoD hardening (all require the 3-of-4 trust root, no attacker reachability): whitelist_beneficiary/toggle not timelocked or pause-guarded; role-distinctness / role-not-beneficiary invariants not enforced in-program; timelock delay bypassable via guardian ack and consensus rotation has no delay/admin-veto; revoke_timelock needs a prior ack (no cancel for un-acked); governance request slots keyable by action byte can be occupied by unexecutable actions.	Add distinctness checks, short additive-whitelist delay/alert window, a non-bypassable minimum rotation delay + admin cancel, and reject impossible action values.
21	zynk-orbit has no global pause/circuit-breaker (Config has no paused field) unlike zynk-core.	Add a pause on the (~\$14K) orbit vault for incident containment.
22	zynk-orbit collect Record account is unconstrained (no PDA seeds), letting the trusted manager stamp an arbitrary whitelisted user_id onto the emitted event.	Constrain the collect record by seeds; reconcilers should key on the authoritative fund fields, not user_id.
23	Order attestation (attest_order) is optional, decorative, replayable, and moves no tokens.	Document it as non-value-authorizing, and if it ever authorizes value off-chain make it mandatory and bind order_id/amount/nonce/expiry.
24	validatePartnerId (utils/index.ts:45) is prefix-only.	Tighten to <code>^zp_[0-9]+(::[a-z]+)?\$</code> (partnerId is SHA-256-hashed into the PDA seed, no injection surface).
25	Mainnet core program IDs are split env-driven vs hardcoded (constants/solana.ts:105-107, Core/index.ts:850-867) though vault PDAs and the live core use the same env value.	Add a startup assertion that the vault-deriving program equals the operating core program.
26	Social-engineering/deepfake/phishing against the 3-of-4 signers and KMS-custody staff.	Enforce out-of-band multi-channel authorization, dedicated hardware signing devices, and endpoint hardening (generic operational advisory, contained by the disjoint 3-of-4 threshold).

Accepted Business / Design Decisions

The following items were reviewed and reclassified as accepted operator tradeoffs rather than findings.

Hot single-key manager for automated ~1-tx/5-min settlement

Automated high-frequency settlement structurally cannot use a human 3-of-4, so a hot automation signer is a knowingly-accepted tradeoff; the actionable defect is the absence of on-chain guardrails (cap/velocity/second-signer), which is tracked as a finding, not the hot key's existence.

Fast-pause / slow-unpause asymmetry in zynk-core

A low-friction single-key emergency halt with a high-assurance multisig+timelock resume is standard, deliberate custody safety design; do not make pause harder to invoke. The single-key pause DoS is recoverable and self-limiting.

close_orders admin force-close skips solvency / canonical-account checks

close_orders is admin-gated (3-of-4 on the funded core), moves no tokens, preserves debt history on-chain, and is exactly the path that must be able to force-close defaulted orders as a write-off; adding a solvency invariant would break the intended cleanup.

Squads create-and-self-approve with MANAGER as an automation member

Enqueuing proposals with one automation vote pre-seeded is an acknowledged operator workflow on a 3-of-4 where two further independent human sign-offs remain; the security gap is the unauthenticated reachability, not the self-approval design.

Config delivered via environment variables (12-factor)

Env-based config for non-secret identifiers (public program IDs, KMS resource names) is standard, widely-accepted custodial practice; only a possible RPC provider key warrants moving into Secret Manager.

AI code review is advisory and label-gated, not an enforced merge gate

Positioning the AI reviewer as advisory with human/process gates enforcing merges is a deliberate process choice; a prompt-injected reviewer cannot approve or merge given its scoped token.

Documentation versus Reality

Claims raised during the engagement compared against what could be verified from the available artefacts.

As Documented	As Verified
EVM decimals bug is exploitable e.g. on BSC (chain 56) and for 18-decimal tokens generally	BSC (56) is not in the supported chain set (1/10/137/8453/42161/59144 + testnets); USDC/USDT are 6-decimal on all supported chains, so only USD1 and RLUSD on Ethereum are actually 18-decimal – the defect is narrower and has no in-unit amount-math consumer.
zynk-orbit collect is unbound 'unlike deposit AND disburse', which bind the record by PDA	In the IDL, disburse's record is also a bare unconstrained writable; the disburse-binding claim rests on an in-handler require! in Rust source not present in this repo (unverified).
The manager key is one of the four members of the Operations multisig	No verification artifact enumerates the Operations multisig members; recon scripts only decode the core Config role addresses – the composition claim is an unverified assumption.
The prompt-injectable AI reviewer threatens the secret-minting deploy runner	The claude-code-review job holds only an Anthropic API key (no OIDC/WIF, no deploy creds); the secret-fetch deploy runner is a different workflow – the risk is misattributed.
Blind-signing cites a 'confirmed WAF bypass' as part of its impact	That clause is a cross-reference to the separate WAF-bypass finding and double-counts it; the blind-signing item is a generic advisory with no independent code/on-chain defect.

Positive Controls Confirmed

The following controls were verified as correctly implemented during the assessment:

- ✓ Sensitive whitelist/toggle/revoke (beneficiary and LP) operations correctly route through the Squads multisig via createTx rather than direct manager signing.
- ✓ The funded core ZYNKctWoa uses disjoint 3-of-4 Squads multisigs for both admin and guardian, holding ~\$2.28M behind a real threshold.
- ✓ All egress paths in zynk-core (create_order, pull_and_create_order, execute_request, replenish) are pause-guarded.
- ✓ disburse enforces an on-chain Record-PDA whitelist check, bounding money movement to already-whitelisted destinations.
- ✓ BYOK/KMS design keeps private key material inside Cloud KMS; signers never export raw keys.
- ✓ Deliberate invariant enforcement is present where it mattered (DuplicateWhitelistedTokenMint error 6016), showing the authors do add on-chain guards.
- ✓ Prod deploy correctly enforces the Jira change-management gate (exit 1) and environment:production, unlike staging.
- ✓ The Claude review workflow uses pull_request (not pull_request_target) with a narrow tool allowlist, so fork PRs receive no secrets or write token.
- ✓ Cosign image signing with digest verification is wired into the deploy pipeline.
- ✓ Snyk code + dependency scanning and AI review are wired as PR checks (defense-in-depth, even if advisory).

Conclusion

In this engagement, QuillAudits assessed the operational security posture of Zynk Protocol, covering the fastlane signing service and SDK, the signer/KMS key-management model, the on-chain zynk-core and zynk-orbit configuration, the GitHub Actions deployment pipelines, the container build, and the Cloudflare/GKE edge infrastructure.

The assessment consolidated 47 raw observations into 10 reportable findings – 2 Critical, 4 High, 2 Medium and 2 Low – with 26 lower-signal items folded into the Minor Hardening list and 6 items reclassified as accepted business/design decisions. The most significant risks centred on the fastlane signer: an unauthenticated, insecurely bound gRPC service whose caller-supplied metadata selected the production MANAGER key, and the executeTx and buildEd25519Ix RPCs, which together acted as arbitrary-instruction and arbitrary-message signing oracles over any KMS key.

All 10 findings are marked Resolved. The arbitrary-input signing primitives (executeTx and buildEd25519Ix) were removed outright; redundant on-chain and off-chain roles were stripped so that the Orbit components now depend on the Core config; the --admin force-merge was removed from the deploy pipelines in favour of required reviewers and enforced dependency and code-scan checks; a .dockerignore and multi-stage Docker build were introduced; and the GKE origins are now restricted to Cloudflare through a Cloud Armor allowlist and Authenticated Origin Pulls.

Four findings were closed on the basis of an architectural rationale rather than a direct code change: retaining mutable z-caller/z-env metadata for environmental routing (#1), the single hot KMS manager key (#4), reliance on on-chain PDA state rather than the cache for settlement idempotency (#6), and the pre-seeded MANAGER vote on Squads proposals (#10). For #4, the team has committed to introducing transaction caps and rate limits in upcoming releases. Because the impact of the hot-key model scales with the funds it controls, we recommend that these guardrails be shipped and independently re-verified before custody is migrated to the V4 core.

We further recommend that the team work through the Minor Hardening list and treat operational security as a continuous practice, with periodic re-assessment as the infrastructure and custody model evolve.

Disclaimer

This operational security assessment was performed by QuillAudits on the source code, configuration, deployment pipelines, on-chain program configuration and externally reachable infrastructure that were made available to, or observable by, the assessment team during the engagement. It reflects the state of those systems at the time of review and the information supplied by ZynkLabs. Changes to code, cloud configuration, network policy, key custody, personnel or access controls made after the assessment may introduce risks that are not covered by this report.

Operational security spans people, processes and technology. Although the review followed industry-standard practice and was carried out thoroughly, an assessment of this kind cannot guarantee the absence of vulnerabilities, misconfigurations or compromise, and cannot account for systems that were not disclosed, insider threats, failures of third-party providers or novel attack techniques. Status markings reflect the remediation state at the time of report finalisation, including fix references and comments supplied by the ZynkLabs team; where a finding was closed through an accepted design decision, the associated residual risk remains with ZynkLabs.

This report is not a security guarantee, an endorsement of Zynk Protocol, or financial, legal or investment advice. It is prepared for ZynkLabs and should not be relied upon by third parties for any decision. We strongly recommend continuous monitoring, periodic re-assessment, independent smart contract audits, incident-response exercises and a public bug bounty programme to sustain resilience against evolving threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

August 2026

For



 QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com